Trabalho avaliativo — Um algoritmo, diferentes linguagens: listas, arrays e repetições

Unidade curricular: Algoritmos e Programação — Alexandre de Oliveira — Montanha

Alunos:

Bernardo Franco Moreira – 126221445077

Gabriel Alves Godoi da Costa – 126221443947

1) 1. Por que utilizamos laços de repetição?

R = Embora o código seja executado sequencialmente de cima para baixo, os laços de repetição permitem rodar um mesmo bloco de código várias vezes sem precisar duplicá-lo manualmente. Eles tornam o programa mais dinâmico e enxuto, sendo ideais para processar listas e arquivos ou manter o sistema ativo continuamente.

Momento da aula: O Montanha mostra a leitura de cima para baixo e a necessidade de quebrar essa linearidade com loops aos 03:01

1) 2. Qual é a diferença entre while e do-while? Criem um exemplo de situação em que essa diferença seja importante.

R = A diferença está no momento da validação. O “while” Testa a condição antes. Pode não executar nenhuma vez se a condição já for falsa. Já o "do-while" executa o código e depois testa a condição. Garante ao menos uma execução, sendo ideal para menus interativos que precisam aparecer antes de o usuário decidir sair.

Exemplo real: Um menu no console onde o usuário escolhe opções. Precisamos exibir o menu e pegar a opção do usuário pelo menos uma vez antes de validar se ele digitou a opção de sair.

Momento da aula: O Montanha mostra essa lógica na prática testando o valor zero na entrada aos 14:10 até 15:52.

1) 3. No exemplo dos times em C#, por que são necessários dois laços?

R = É necessários dois laços porque precisamos cruzar os dados de um elemento com todos os outros da mesma lista. O primeiro laço fixa o time que joga em casa da partida, e o segundo laço corre a lista inteira de novo para associar cada um dos possíveis visitantes. Sem esses dois laços rodando juntos, a gente não conseguiria montar todas as combinações de jogos.

1) 4. Qual é a função do if que impede confrontos de um time contra ele mesmo?

R = Como os dois laços usam a mesma lista partindo do início, fatalmente o laço de dentro vai cair no mesmo índice do laço de fora *(i = j)*. Se não fosse colocado esse *if (i != j)*, o programa ia tentar agendar jogos impossíveis como "Maior de minas x Maior de minas" (Atlético x Atlético). O *if* serve justamente para descartar essas partidas sem nexo.

1) 5. Comparando os exemplos de Java, C# e Rust, indiquem duas diferenças de sintaxe e um conceito lógico que permanece igual.

R =

Diferença 1 (Declaração): Em Java e C# precisamos dizer o tipo logo no início da variável (String[] ou List<string>), enquanto em Rust usamos let mut e a macro vec![], indicando explicitamente se a variável pode ser alterada ou não.

Diferença 2 (Sintaxe do laço): Java e C# usam o for clássico com três partes separadas por ponto e vírgula (for (int i = 0; i < n; i++)), enquanto Rust adota uma sintaxe bem mais enxuta usando intervalos diretamente (for i in 0..times.len()).

Conceito igual: A estrutura lógica do problema. Nas três linguagens, a gente precisou usar dois loops aninhados para cruzar os times e uma condição para não deixar um time jogar contra si mesmo.

2) 1. Como os elementos de um array são organizados e acessados?

R = Geralmente eles ficam guardados na memória um grudado no outro, em posições contínuas. A grande vantagem disso é que o computador sabe exatamente o tamanho de cada bloco e onde a lista começa. Com isso, ele pula direto para o índice que você deseja instantaneamente, sem precisar passar pelos itens anteriores.

2) 2. Como os elementos de uma lista encadeada são organizados? Apresentem um desenho simples.

R = Eles não necessariamente precisam ficar juntos na memória, podem estar espalhados em qualquer canto livre da memória RAM. O que mantém a lista unida é que cada item guarda o seu próprio dado e um endereço (ponteiro) apontando para onde está o próximo elemento.

["Atlético-MG" | aponta pro próx] —> ["Cruzeiro" | aponta pro próx] —> ["América-MG" | null]

2) 3. Por que inserir ou excluir elementos pode exigir operações diferentes nessas duas estruturas?

R = No array, o tamanho dele é sempre fixo. Caso você quiser colocar mais alguém e a memória do lado estiver preenchida, o sistema tem que mover o array inteiro para outro canto com mais espaço livre. Além disso, colocar ou tirar alguém do meio obriga a empurrar todos os outros itens para o lado. Na lista encadeada isso é bem mais tranquilo: você só precisa alterar os ponteiros do item ao lado para incluir ou tirar alguém da fila, sem precisar mexer no resto da memória.

2) 4. Uma coleção chamada “lista” em uma linguagem é necessariamente uma lista encadeada? Pesquisem especificamente List<string> de C# e Vec<String> de Rust.

R = Não necessariamente. Estruturas como o List<string> do C# e o Vec<String> do Rust são, por baixo dos panos, arrays dinâmicos. Elas mantêm os itens em memória contínua para você acessar rápido pelo índice. A diferença é que elas escondem a parte chata: quando o espaço acaba, elas mesmas alocam um array maior e copiam os dados de forma automática. Listas encadeadas de verdade no C#, por exemplo, usam uma classe própria chamada LinkedList<T>.

2) 5. Para guardar os times e percorrê-los repetidamente, quais vantagens vocês identificam em uma estrutura baseada em array?

R = A maior vantagem é a agilidade para buscar os times direto pela posição (times[i] e times[j]), já que o acesso é imediato ($O(1)$). Além disso, como os itens ficam lado a lado na memória física, o processador consegue puxar blocos inteiros da lista direto para a memória cache (cache locality), tornando as passagens dos loops muito mais rápidas do que se tivéssemos que pular de ponteiro em ponteiro pela memória RAM.

Para responder as perguntas das seções, foram utilizadas, principalmente, as explicações apresentadas na videoaula disponibilizada, bem como os slides disponíveis no Ulife.

3) Códigos disponíveis em:

https://github.com/bernardomfb1/Trabalho-avaliativo-Um-algoritmo-diferentes-linguagens-listas-arrays-e-repeti-es.git

4) 1. Por que o array de Java exigiu a criação de outro array?

R = O array primitivo no Java tem um tamanho fixo definido no momento da sua criação. Não é possível expandi-lo na memória, forçando a instanciação de um novo array de tamanho 5 e a cópia manual dos elementos do array antigo.

4) 2. O que ficou mais simples com List<string> e Vec<String>?

R = Nas duas foi reduzida a uma única instrução, eliminando a necessidade de gerir manualmente o tamanho da estrutura ou criar laços de cópia. No C#, bastou chamar o método times.Add("Tombense");. Da mesma forma, no Rust a adição foi concluída facilmente com a instrução times.push(String::from("Tombense")).

4) 3. A facilidade de adicionar um elemento significa que essas estruturas são listas encadeadas? Justifiquem com a pesquisa realizada.

R = Não. Apesar da simplicidade nos métodos, eles operam como arrays dinâmicos na memória. Eles mantêm os dados em blocos contíguos, mas a linguagem automaticamente fica responsavel de alocar novos espaços e copiar os dados em segundo plano sempre que a capacidade atual é excedida, escondendo essa complexidade.

5) 1. TABELA

R =

Operação	Java	C#	Rust
Criar a coleção	String[] times = {"Atlético-MG", "Cruzeiro", "América-MG", "Villa Nova"};	List<string> times = new List<string> { "Atlético-MG", "Cruzeiro", "América-MG", "Villa Nova" };	let mut times: Vec<String> = vec![String::from("Atlético-MG"), String::from("Cruzeiro"), String::from("América-MG"), String::from("Villa Nova")];
Consultar a quantidade de elementos	times.length	times.Count	times.len()
Acessar um elemento pelo índice	times[i]	times[i]	times[i]
Escrever um laço para percorrer a coleção	for (int i = 0; i < times.length; i++)	for (int i = 0; i < times.Count; i++)	for i in 0..times.len()
Escrever dois laços aninhados	for (int i = 0; i < times.length; i++) { for (int j = 0; j < times.length; j++) { ... } }	for (int i = 0; i < times.Count; i++) { for (int j = 0; j < times.Count; j++) { ... } }	for i in 0..times.len() { for j in 0..times.len() { ... } }
Impedir confronto contra o próprio time	if (i != j)	if (i != j)	if i != j
Acrescentar o quinto time	String[] novoArray = new String[times.length + 1]; // ... cópia dos elementos ... novoArray[novoArray.length - 1] = "Tombense"; times = novoArray;	times.Add("Tombense");	times.push(String::from("Tombense"));

5) 2. O que mudou na escrita dos programas e o que permaneceu igual no raciocínio utilizado para resolver o problema?

R = A escrita dos programas mudou na declaração de variáveis, métodos de adição de elementos e na forma (ex: .length em Java, .Count em C#, .len() em Rust).

O raciocínio foi o mesmo nas três linguagens, uma vez que utilizaram dois laços para percorrer, usando i != j para evitar jogos entre o mesmo time na ida e na volta e ajustar o limite do segundo para apenas um turno.

6) 1.

primeiro time (i)	segundo time (j)	Combinação examinada	Exibe o jogo? Por quê?
0	0	0 x 0	Não. A condição (i != j) é falsa, pois os índices são iguais.
0	1	0 x 1	Sim. A condição (i != j) é verdadeira.
0	2	0 x 2	Sim. A condição (i != j) é verdadeira.
1	0	1 x 0	Sim. A condição (i != j) é verdadeira.
1	1	1 x 1	Não. A condição (i != j) é falsa, pois os índices são iguais.
1	2	1 x 2	Sim. A condição (i != j) é verdadeira.
2	0	2 x 0	Sim. A condição (i != j) é verdadeira.
2	1	2 x 1	Sim. A condição (i != j) é verdadeira.
2	2	2 x 2	Não. A condição (i != j) é falsa, pois os índices são iguais.

6) 2. Quantidade de times Jogos de ida e volta Jogos em turno único
3
4
5

R =

Quantidade de times	Jogos de ida e volta	Jogos em turno único
3	6	3
4	12	6
5	20	10

6) 3.1. Por que a quantidade de combinações examinadas pode ser maior que a quantidade de jogos exibidos?

R = Porque os laços executam a iteração cruzando todas as possibilidades, incluindo a

mesma equipe em ambos os índices ex (índice 0 com índice 0). A variável de controle só é adicionada quando a condição (if i != j) é seguida, descartando assim as situações inválidas ("Maior de minas x Maior de minas" (Atlético x Atlético))

6) 3.2. O que acontece com a quantidade de confrontos quando acrescentamos um time?

R = A quantidade cresce consideravelmente. Ao verificar na tabela acima, a tabela dos 4 times para os 5 times (incluindo o "Tombense"), os jogos de ida e volta aumentaram de 12 para 20, e os jogos de turno único de 6 para 10.

6) 3.3. As três linguagens produziram os mesmos jogos e totais? Caso contrário, identifiquem e corrijam a diferença.

R = Sim, as variáveis de totalIdaEVolta e totalTurnoUnico operam sob a mesma lógica, produzindo sempre os mesmos resultados numéricos.

7) 1. Evidências de execução das três versões

R = Os programas foram executados com sucesso nos terminais Java, .NET e Cargo. Em todas as linguagens, o final com 5 equipes gerou exatamente os mesmos resultados: 20 jogos de ida e volta, e 10 jogos em turno único.

Prints dos resultados nas próximas páginas:

TESTE EM JAVA

```
=== TABELA DOS 4 INICIAIS TIMES ===

Times:
[0] Atlético-MG
[1] Cruzeiro
[2] América-MG
[3] Villa Nova

Ida e Volta:
Jogo 1: Atlético-MG x Cruzeiro
Jogo 2: Atlético-MG x América-MG
Jogo 3: Atlético-MG x Villa Nova
Jogo 4: Cruzeiro x Atlético-MG
Jogo 5: Cruzeiro x América-MG
Jogo 6: Cruzeiro x Villa Nova
Jogo 7: América-MG x Atlético-MG
Jogo 8: América-MG x Cruzeiro
Jogo 9: América-MG x Villa Nova
Jogo 10: Villa Nova x Atlético-MG
Jogo 11: Villa Nova x Cruzeiro
Jogo 12: Villa Nova x América-MG
Total (Ida e Volta): 12

Turno Único:
Jogo 1: Atlético-MG x Cruzeiro
Jogo 2: Atlético-MG x América-MG
Jogo 3: Atlético-MG x Villa Nova
Jogo 4: Cruzeiro x América-MG
Jogo 5: Cruzeiro x Villa Nova
Jogo 6: América-MG x Villa Nova
Total (Turno Único): 6

=== TABELA DOS 5 FINAIS TIMES ===

Times:
[0] Atlético-MG
[1] Cruzeiro
[2] América-MG
[3] Villa Nova
[4] Tombense

Ida e Volta:
Jogo 1: Atlético-MG x Cruzeiro
Jogo 2: Atlético-MG x América-MG
Jogo 3: Atlético-MG x Villa Nova
Jogo 4: Atlético-MG x Tombense
Jogo 5: Cruzeiro x Atlético-MG
Jogo 6: Cruzeiro x América-MG
Jogo 7: Cruzeiro x Villa Nova
Jogo 8: Cruzeiro x Tombense
Jogo 9: América-MG x Atlético-MG
Jogo 10: América-MG x Cruzeiro
Jogo 11: América-MG x Villa Nova
Jogo 12: América-MG x Tombense
Jogo 13: Villa Nova x Atlético-MG
Jogo 14: Villa Nova x Cruzeiro
Jogo 15: Villa Nova x América-MG
Jogo 16: Villa Nova x Tombense
Jogo 17: Tombense x Atlético-MG
Jogo 18: Tombense x Cruzeiro
Jogo 19: Tombense x América-MG
Jogo 20: Tombense x Villa Nova
Total (Ida e Volta): 20

Turno Único:
Jogo 1: Atlético-MG x Cruzeiro
Jogo 2: Atlético-MG x América-MG
Jogo 3: Atlético-MG x Villa Nova
Jogo 4: Atlético-MG x Tombense
Jogo 5: Cruzeiro x América-MG
Jogo 6: Cruzeiro x Villa Nova
Jogo 7: Cruzeiro x Tombense
Jogo 8: América-MG x Villa Nova
Jogo 9: América-MG x Tombense
Jogo 10: Villa Nova x Tombense
Total (Turno Único): 10
```

TESTE EM C#

```
PS C:\Users\Bernardo\Desktop\Faculdade\Trabalho - Campeonato - Java - C# - Rust\geradorDeConfrontos - C#> & 'c:\Users\Bernardo\.vscode\extensions\ms-dotnettools
.160.4-win32-x64\.debugger\x86_64\vsdbg.exe' '--interpreter=vscode' '--connection=05753b96398f4cd799a8a0ad02a8cc55'
=== 4 TIMES ===

Times:
[0] Atlético-MG
[1] Cruzeiro
[2] América-MG
[3] Villa Nova

Ida e Volta:
Jogo 1: Atlético-MG x Cruzeiro
Jogo 2: Atlético-MG x América-MG
Jogo 3: Atlético-MG x Villa Nova
Jogo 4: Cruzeiro x Atlético-MG
Jogo 5: Cruzeiro x América-MG
Jogo 6: Cruzeiro x Villa Nova
Jogo 7: América-MG x Atlético-MG
Jogo 8: América-MG x Cruzeiro
Jogo 9: América-MG x Villa Nova
Jogo 10: Villa Nova x Atlético-MG
Jogo 11: Villa Nova x Cruzeiro
Jogo 12: Villa Nova x América-MG
Total (Ida e Volta): 12

Turno Único:
Jogo 1: Atlético-MG x Cruzeiro
Jogo 2: Atlético-MG x América-MG
Jogo 3: Atlético-MG x Villa Nova
Jogo 4: Cruzeiro x América-MG
Jogo 5: Cruzeiro x Villa Nova
Jogo 6: América-MG x Villa Nova
Total (Turno Único): 6

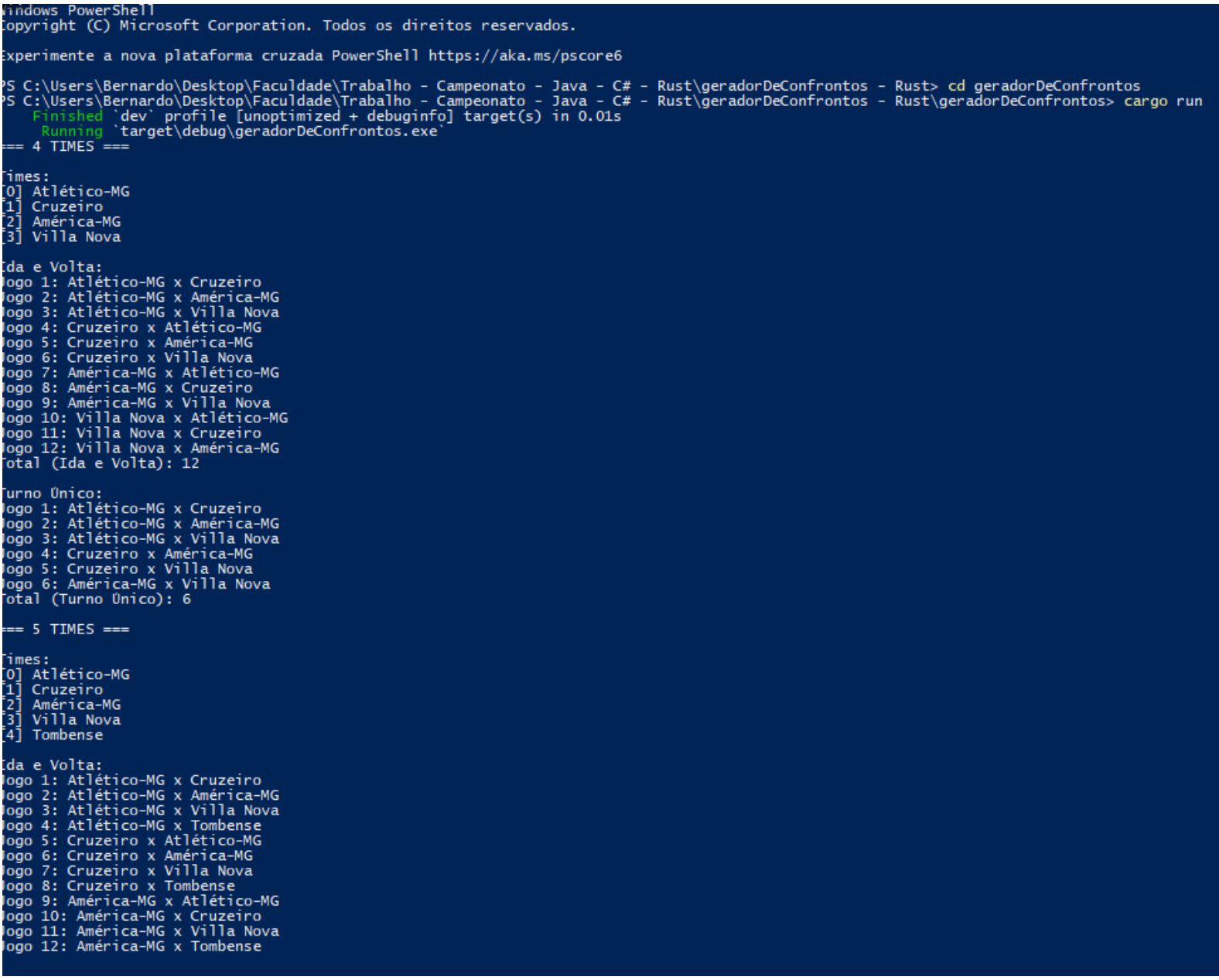
=== 5 TIMES ===

Times:
[0] Atlético-MG
[1] Cruzeiro
[2] América-MG
[3] Villa Nova
[4] Tombense

Ida e Volta:
Jogo 1: Atlético-MG x Cruzeiro
Jogo 2: Atlético-MG x América-MG
Jogo 3: Atlético-MG x Villa Nova
Jogo 4: Atlético-MG x Tombense
Jogo 5: Cruzeiro x Atlético-MG
Jogo 6: Cruzeiro x América-MG
Jogo 7: Cruzeiro x Villa Nova
Jogo 8: Cruzeiro x Tombense
Jogo 9: América-MG x Atlético-MG
Jogo 10: América-MG x Cruzeiro
Jogo 11: América-MG x Villa Nova
Jogo 12: América-MG x Tombense
Jogo 13: Villa Nova x Atlético-MG
Jogo 14: Villa Nova x Cruzeiro
Jogo 15: Villa Nova x América-MG
Jogo 16: Villa Nova x Tombense
Jogo 17: Tombense x Atlético-MG
Jogo 18: Tombense x Cruzeiro
Jogo 19: Tombense x América-MG
Jogo 20: Tombense x Villa Nova
Total (Ida e Volta): 20

Turno Único:
Jogo 1: Atlético-MG x Cruzeiro
Jogo 2: Atlético-MG x América-MG
Jogo 3: Atlético-MG x Villa Nova
Jogo 4: Atlético-MG x Tombense
Jogo 5: Cruzeiro x América-MG
Jogo 6: Cruzeiro x Villa Nova
Jogo 7: Cruzeiro x Tombense
Jogo 8: América-MG x Villa Nova
Jogo 9: América-MG x Tombense
Jogo 10: Villa Nova x Tombense
Total (Turno Único): 10
```

TESTE EM RUST (RODAMOS NO POWERSHELL)



7) 2. Um erro encontrado e como foi corrigido.

R = Ao desenvolver a versão em Java, cometemos um erro clássico no limite do laço de repetição: escrevemos i <= times.length em vez de i < times.length. Isso causou o erro no terminal, pois o programa tentou um índice que não existia. Correção: alteramos a condição para apenas menor, garantindo que o laço para na última posição válida.

7) 3. Um parágrafo de cada integrante explicando sua contribuição e seu principal aprendizado.

R = Bernardo: Fiquei responsável por estruturar o código base em Java e tentar passar a lógica dos laços aninhados para o C#. O meu principal aprendizado foi tentar entender, na prática, como o List<string> do C# facilita em comparação com os arrays de tamanho fixo do Java, poupando o desenvolvedor. Nessa atividade reforcei o pensamento de focar em uma logica de programação boa e não me apegar a uma linguagem de programação.

Godoi (Gabriel): Fiquei com a responsabilidade de configurar o ambiente Rust, programar a respectiva versão e elaborar os testes de mesa. O meu principal aprendizado foi compreender a sintaxe do Rust (como o macro vec!) e consolidar a ideia de que a lógica dos laços (j = i + 1 para turno único) é universal, independentemente da linguagem.

7) 4. Referências consultadas.

R =

1. Videoaula do Montanha.
2. Slides disponíveis, tanto do Montanha quanto do Flávio.
3. Para as linguagens utilizamos a documentação disponível nos sites abaixo:
   C#: https://learn.microsoft.com/pt-br/dotnet/csharp/
   Java: https://docs.oracle.com/javase/tutorial/java/
   Rust: https://doc.rust-lang.org/stable/book/
4. Foi utilizado o Notebook LM do Gemini para nos ensinar também em partes as estruturas nas linguagens C# e Rust, uma vez que havia sido nosso primeiro contato com as linguagens. Buscamos tentar fazer mais ou menos como usamos o Portugol, codamos em Java e tentamos buscar entender o que tal linha de código poderia ser montada nas outras linguagens, como exemplo da variável, montamos em Java mas nas documentações buscamos procurar como se comportaria nas outras linguagens.